

QuantAnomalies

Quantitative Anomalies as Market Opportunities

Progress Presentation • Statistical & AI Dashboard for Equity and Forex Markets

1 Title of the Study

Quantitative Anomalies as Market Opportunities — A Statistical & AI Dashboard for Equity and Forex Markets.

In plain terms: textbook finance says markets are “efficient” — prices already reflect everything, risk never changes, and nothing is predictable. Reality is messier, and those cracks in the theory are recurring patterns called **anomalies**. Our project hunts for those anomalies and reframes them as **opportunities**: places where the odds may tilt in a careful observer’s favour.

2 Objectives of the Study

Each objective targets one well-known market anomaly and turns it into a usable opportunity signal.

1. **Macro-driver opportunity.** Measure *which* macroeconomic forces (interest rates, inflation, the fear index, oil, gold, the dollar) move an asset’s returns — and crucially with *what time delay*. If a driver acts weeks later, that lag is a window to anticipate moves.
2. **Risk-regime opportunity.** Model how risk itself rises and falls over time (volatility clustering) and detect that crashes happen far more often than a bell curve predicts (fat tails). Spotting a “stormy” regime early is a chance to cut risk — or to recognise unusually cheap calm.
3. **Mean-reversion opportunity.** Find pairs of currencies whose prices are tethered over the long run (cointegration), then flag when they drift unusually far apart — history says the gap tends to snap back. This is the basis of market-neutral “statistical arbitrage”.
4. **Opportunity synthesis (the decision engine).** Combine all of the above into one automated scanner that *fuses* what the three pillars detect into a single directional verdict — with agreement, conviction, and disagreement all made explicit — and uses a **local AI analyst** to explain it in plain English.
5. **Accessibility & robustness.** Make it work for *any* ticker the user types, on a reliable data pipeline, presented through a clear, interactive dashboard.

3 Motivation for Selecting the Topic

- **The anomalies are real and well-studied, but rarely made tangible.** Lagged macro effects, volatility clustering, fat tails, and cointegration all appear in finance literature; we wanted to make them *visible and actionable* rather than abstract equations.
- **Reframing risk as opportunity.** The same statistics used to warn about risk can also point to opportunity — we wanted a tool that does both.

- **Bridging statistics and modern AI.** We combine classic econometrics (regression, GARCH, cointegration) with a **local large language model** that turns numbers into a readable analyst note — private, free to run, and on our own GPU.
- **Learning value.** The project spans real statistical modelling, live financial-data engineering, full-stack web development, and on-device AI.

4 Data & APIs Used

In place of a questionnaire and a survey sample: our “instrument” is a set of financial-data APIs, and our “sample” is the live market history they return.

A. External data-source APIs (collecting raw market data)

- **Yahoo Finance** (via the `yfinance` library) — daily price history (2015–2025) for any equity, index, currency pair, future, or crypto. Examples: `^GSPC` (S&P 500), `^VIX` (volatility index), `CL=F` (oil), `GC=F` (gold), `^TNX` (10-year Treasury yield), `DX-Y.NYB` (US dollar index), and 45 forex pairs.
- **FRED — U.S. Federal Reserve Economic Data** — macroeconomic series: CPI (inflation), Fed Funds Rate, and Unemployment. A free mirror, **DBnomics**, is a built-in backup so a single outage never stops the study.

B. Our own analysis API (FastAPI backend)

Turns raw data into results:

- `/api/macro-regression/*` — regression, Granger-causality, correlation, time series
- `/api/garch/*` — volatility model, clustering tests, return distribution
- `/api/pairs/*` — cointegration tests, best-pair spread & signals, correlation
- `/api/options/black-scholes` — option price, Greeks, implied volatility
- `/api/engine/{feed,asset,narrate,status}` — the fused opportunity scan, the streaming AI narration, and the engine’s health
- `/api/llm/info` — status of the local AI analyst

C. Local AI API

An on-device `llama.cpp` server (OpenAI-compatible) running the **Qwen3-4B** model on our **NVIDIA RTX 4050 GPU**, used to explain detected opportunities in plain language. No data leaves the machine.

5 Pilot Study Conducted to Date

We have a **working end-to-end prototype**, validated on real data. *All figures below were recomputed on 26 July 2026.*

Data pipeline — and one real finding. We had been treating a data corruption as a bad cache. The actual root cause is that **yfinance is not thread-safe**: concurrent downloads silently return one ticker’s data under another’s name. A deliberately threaded scan reproduced it on demand — three separate ticker pairs came back byte-identical. Downloads are now serialized. Separately, the data end-date had been frozen, so the dashboard was reasoning over seven-month-old prices; now a rolling window with a staleness check.

Three analysis pillars — working, with sample findings on the S&P 500:

- *Macro*: standardized macro factors explain about **69%** of monthly return variation ($R^2 = 0.687$, adj. 0.585); Granger causality runs 32 tests, **6 significant**. Coefficients are standardized betas, so "which driver matters most" is a fair comparison.
- *Volatility*: GARCH persistence $\approx$ **0.994** over 2,905 days (shocks fade very slowly); excess kurtosis $\approx$ **15.8**, skew **-0.65**, Jarque-Bera $p \approx 0$ — risk is *not* constant, and the normal curve badly under-states crashes.
- *Pairs*: **USDCHE/USDJPY** tested cointegrated ($p = 0.0075$) with a mean-reversion half-life of **69 days**, on log prices.

Decision engine — the headline advance. Seven detectors across all three pillars, normalized onto one bull/bear axis and then **fused**: weight = severity $\times$ statistical reliability, producing a direction (tilt) and a conviction that *falls* when signals disagree, with risk on a separate axis so a volatility spike cannot fake a directional call. Signals that oppose the verdict are shown, not averaged away. Warm response: **11 ms** for the cross-asset feed, **4.5 ms** per asset.

The AI layer, kept honest. The model is handed the computed numbers and explains them; it never computes. Its own stance renders *beside* the computed one, so a disagreement is visible. Every figure in its note is checked against the supplied evidence and anything unmatched is flagged in the UI. ≈ 4 seconds on the GPU (≈ 28 tokens/sec), streaming, fully local.

Dashboard (working). Any ticker, any forex pairs, time ranges on every chart, five tabs (Overview, Opportunities, Macro, GARCH, Pair Trading), light/dark themes with all colour pairings verified at $\geq 4.5:1$ contrast.

In short: the pilot confirms the data, the statistics, the fusion layer, the AI explanation, and the interface all work together on live markets. The honest remaining gap is **inferential rigor** — multiple-testing correction on the 990 cointegration tests, HAC standard errors on the autocorrelated monthly regression, and an out-of-sample check on the signals. That is the next block of work.

6 Modules Built So Far

Backend (Python / FastAPI)

- `data_loader.py` — data acquisition (Yahoo Finance, FRED, DBnomics), caching, dataset builders
- `analysis/macro_regression.py` — Pillar 1: OLS lag regression, Granger causality, correlation
- `analysis/garch.py` — Pillar 2: GARCH(1,1), volatility clustering, return distribution
- `analysis/pairs.py` — Pillar 3: cointegration, spread / z-score, trading signals
- `analysis/recommender.py` — the seven detectors over the pillar outputs
- `analysis/black_scholes.py` — option pricing, Greeks, implied-volatility solver
- `engine.py` — the decision engine: polarity normalisation, reliability weighting, fusion, the tiered/cached scan, and streaming narration
- `llm_client.py` — provider-agnostic local-LLM client (GPU server + CPU fallback)
- `main.py` — FastAPI endpoints tying it all together
- `test_engine.py` — 25 tests over polarity, fusion invariants, ranking, reliability, stance parsing, and the number guardrail

Frontend (React + Vite)

- Five tab panels — Overview, Opportunities, Macro Regression, GARCH Volatility, Pair Trading
- Shared components — SVG icon set, info-tooltips, time-range filter, ticker search, forex selector, status states, tilt gauge, skeletons
- Support — design tokens, light/dark theme hook, shared ticker context, SSE narration hook, number formatting, data-fetch hook with abort + session cache

7 Future Plans — Expanding the Opportunity Engine

Each future module is another *lens* for spotting opportunity. The engine's principle stays the same: **statistics detect, the AI explains.**

1. Options & Black–Scholes (module built; detector not yet wired)

Price European call / put options and their Greeks, and back out the market's **implied volatility**. The module and its endpoint exist and already compute a verdict; what remains is feeding that verdict into the engine as a signal. **Opportunity:** compare implied volatility with our GARCH forecast — when the market pays far more (or less) for risk than the model expects, options are “rich” or “cheap”.

2. Valuation — “is the stock bloated / overpriced?”

Pull fundamental ratios (P/E, P/B, P/S, PEG, EV/EBITDA) and compare them with the stock's own history and its peers, alongside a technical “stretch” check (how far price sits above its 200-day average, and RSI). **Opportunity:** flag names “priced for perfection” (avoid / potential short) versus those “on sale” (potential value buys).

3. More opportunity lenses

- **Momentum & relative strength** — rank a universe by risk-adjusted 12–1 month momentum to spot rotation leaders and laggards.
- **Oversold bounce** — distance from the 52-week high plus RSI to find mean-reversion candidates.
- **Volatility squeeze** — detect compression that often precedes a breakout.
- **Correlation-regime shift** — assets that usually move together decoupling (a diversification or pairs opportunity).
- **Seasonality / calendar effects** — turn-of-month and “sell in May” style documented anomalies.
- **Alpha vs beta** — separate market-driven moves from stock-specific mispricing.
- **Unusual volume** — accumulation / distribution versus the average.
- **Risk budgeting** — turn the detected regime into a suggested position size.

8 Decision Engine: Rules-based vs LLM-based

We combine them: *rules decide, the LLM explains*. Rules alone is the default and works with no model running. Where the two meet, the design keeps the seam visible — the model's stance renders beside the computed stance so it may disagree openly, and every figure it writes is checked against the evidence it was given.

	Rules-based	LLM-based
Determinism	Fully deterministic, reproducible	Varies with settings
Explainability	Transparent thresholds	Natural-language reasoning
Numbers	Exact by construction	Must be guarded against errors
Flexibility	Rigid; new rules need code	Synthesises novel combinations
Speed & cost	Instant, free	≈4 s on GPU, compute cost
Works offline	Yes	Needs the local model
Best at	Detection & signals	Explanation & synthesis

Our design uses each for what it is good at — rules produce the trustworthy numbers, the LLM produces the readable story. A future **LLM decision-maker** mode would let the model also weigh and rank signals (not just explain), kept honest by rule-based guardrails and number validation.

Appendix — Each anomaly in one line (layman)

- **Lagged macro effect:** *News ripples through markets over weeks, not instantly — know the lever and the delay, and you can position ahead.*
- **Volatility clustering:** *Calm days cluster, and so do wild days — storms arrive in groups, giving you warning to manage risk.*
- **Fat tails:** *Extreme crashes happen much more often than the textbook bell curve says — plan for them.*
- **Cointegration / pairs:** *Two currencies on one leash: when they drift far apart, they usually snap back — bet on the gap closing, up market or down.*